🎃 Spookifier Walkthrough — From Curiosity to Code Execution

The Challenge

While working through a web challenge on Hack The Box, I came across a simple-looking application called **“Name Spookifier.”**

The instructions were straightforward:

- Start the challenge
- Get the target IP
- Visit the web app at: `http://<target-ip>`

The page itself looked harmless—a Halloween-themed UI where you could enter your name and get a “spooky” version of it.

At first glance, nothing screamed vulnerability. Just a textbox and a button labeled **“Spookify.”**

But in CTFs, “simple” usually means *look closer*.

The Discovery

I started with basic input:

```
Hi
```

It worked as expected—the app returned a styled output. Nothing unusual.

Then curiosity kicked in.

What happens if I input numbers instead of text?

I tried:

```
7
```

Still normal output.

Then I pushed a bit further:

```
7*8
```

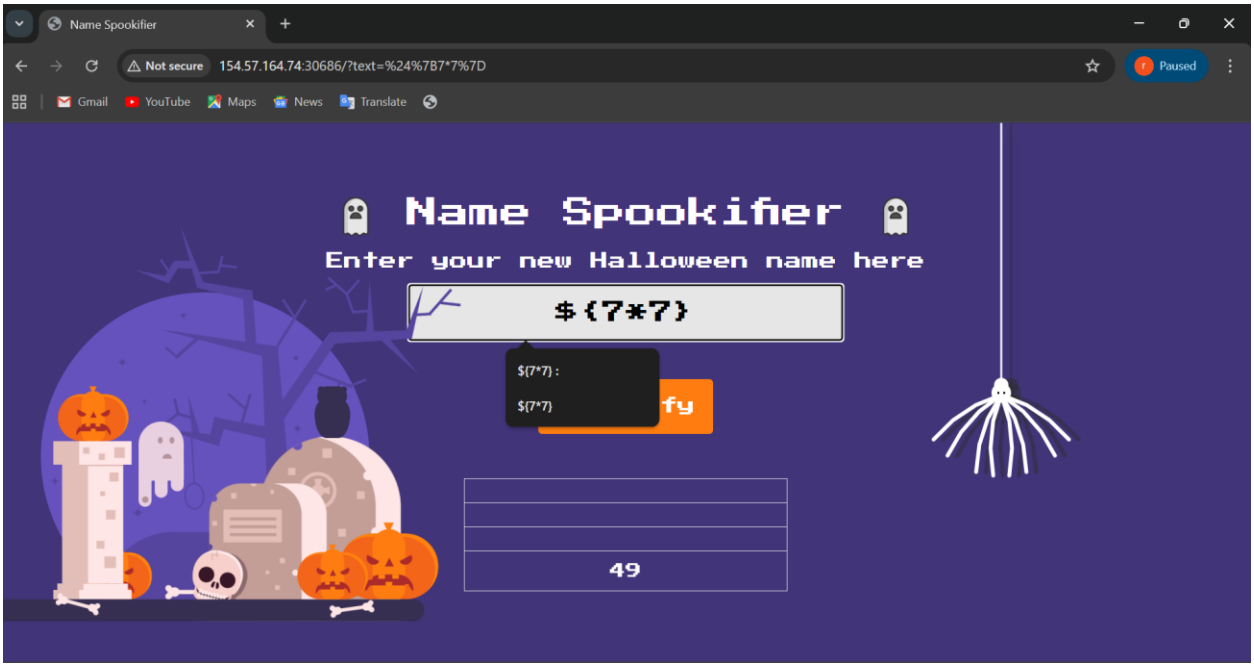
Surprisingly, the result didn't change in any meaningful way. That made me wonder…

Is this being evaluated somewhere behind the scenes?

So I tried a classic test for template injection:

```
${7*7}
```

And there it was.



Output: `49`

That single response changed everything.

This wasn't just a text formatter — it was evaluating expressions server-side.

The Hack

Now that I suspected **Server-Side Template Injection (SSTI)**, the next step was to see how far I could push it.

Step 1: Basic Command Attempt

I tried:

```
${whoami}
```

But it threw an error.

So direct execution wasn't allowed—but that didn't mean it was impossible.

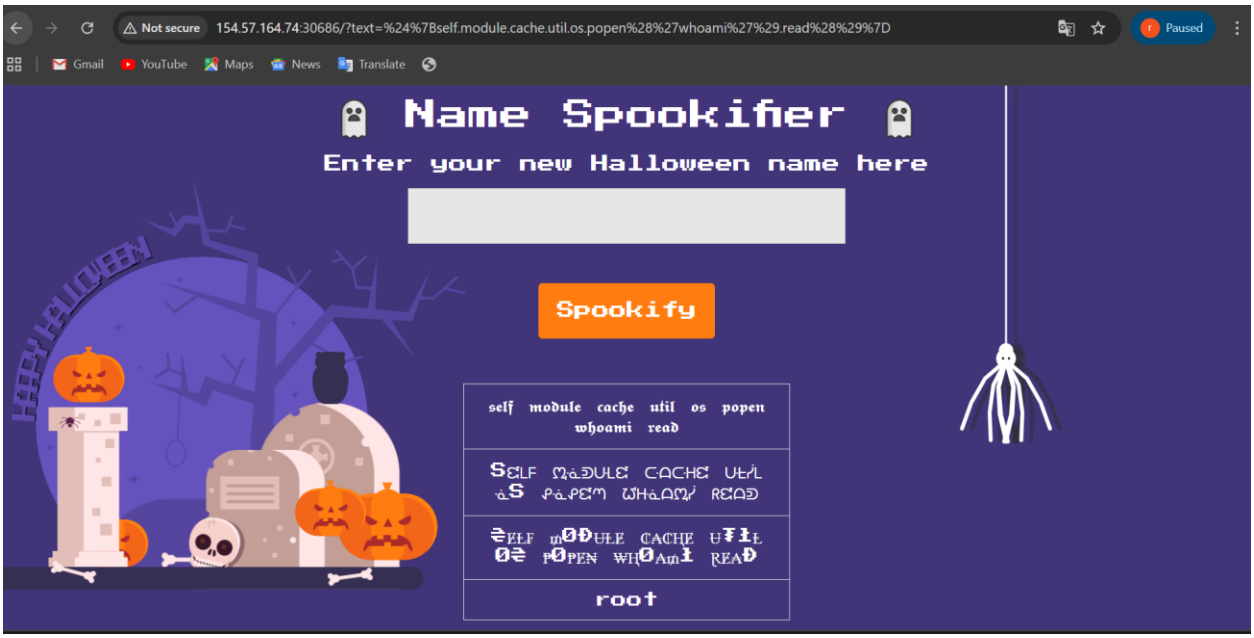
Step 2: Digging Deeper

At this point, I turned to payload techniques inspired by common SSTI exploitation approaches (like those documented in OWASP resources and community payload collections).

Instead of calling commands directly, I tried accessing Python internals:

```
${self.module.cache.util.os.popen('whoami').read()}
```

This worked.



The application returned the system user.

That confirmed it:

- The backend was using Python
- I had access to OS-level command execution

Step 3: Going for the Flag

In CTF challenges, once you have command execution, the goal is usually clear—find the flag.

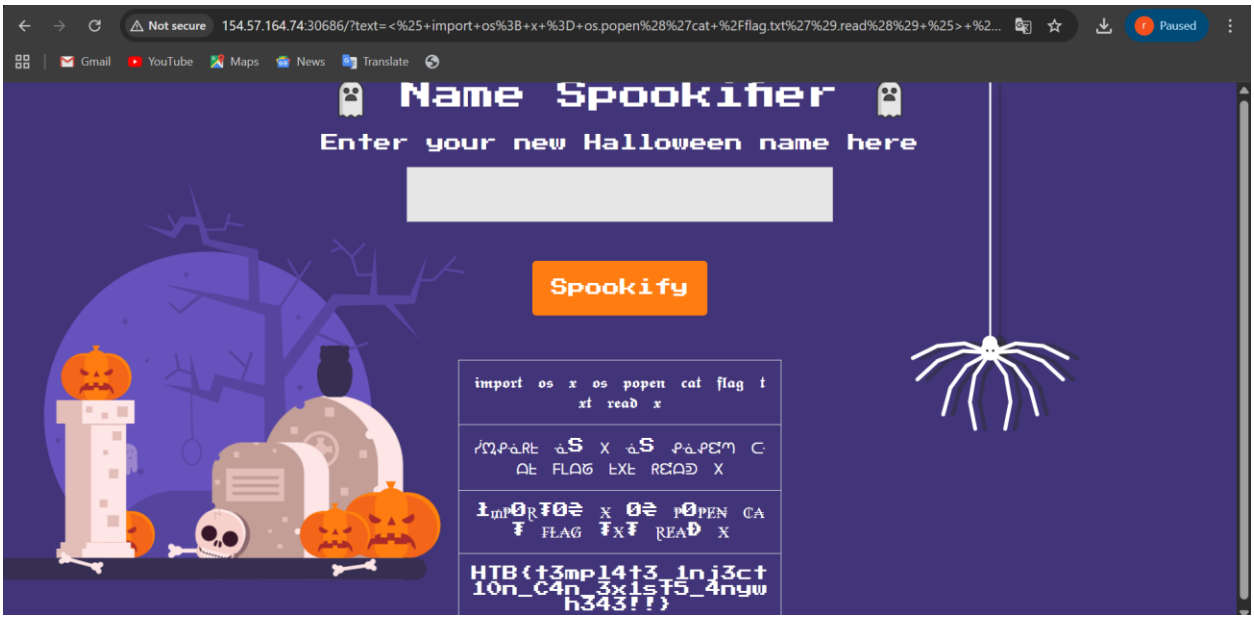
From the provided context, I knew the flag might be stored in:

```
/flag.txt
```

So I crafted a payload using Python import syntax:

```
<% import os; x = os.popen('cat /flag.txt').read() %> ${x}
```

And just like that…



🎯 The contents of `flag.txt` were displayed on the page.

Game over.

The Lesson

This challenge is a perfect example of how **small oversights can lead to critical vulnerabilities**.

Key Takeaways

- **User input should never be trusted**
  Rendering raw input inside templates is dangerous.
- **SSTI is powerful**
  What starts as `${7*7}` can quickly escalate to full **Remote Code Execution (RCE)**.
- **Error messages matter**
  Even failed payloads (like `${whoami}`) provide clues about the backend.
- **Know your tools and payloads**
  Understanding how template engines interact with underlying languages (like Python) is key to exploitation.

Final Thought

What made this challenge interesting wasn't complexity—it was **curiosity**.

A simple question:

"What happens if I try this?"

…led step-by-step to full system compromise.

And that's often how real-world vulnerabilities are discovered too.